1/7/2024

Introduction to HTML and Css

↳ HTML (Hyper-text Markup Language) is the standard markup Language used to create and structure Content on webpages. It defines the basic structure of webpage using elements and tags, such as headings, paragraphs, links, images. Lists, tables, forms and other multimedia Content. HTML documents are interpreted by Web browsers to display the content in a structured and meaningful manner. HTML Provides the structure and Semantic Organization of a webpage

↳ Css (Cascading Style sheets) is a stylesheet language used to control the Presentation and the visual appearance of HTML documents. It is used to define properties such as Colors, fonts, backgrounds, spacing, borders, alignment, dimensions and Layouts. Css also Supports responsive design, allowing webpages to adapt to different screen sizes and devices ex Can be applied using inline, internal or external Stylesheets.

↳ HTML and CSS work together to develop web pages where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of web development and enables the creation of aesthetically attractive and user-friendly web interfaces.

Difference between MERN Stack and Full Stack

↳ MERN stack is a Javascript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js. MongoDB is used for database management, Express.js handles the backend framework and APIs, React.js is used to develop frontend user interface, and Node.js provides the server-side runtime environment.

↳ Full-stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

HTML Components:

1) HTML Tags :- Define the structure and elements of a webpage.

2) Headings :- Used to display titles and headings using <h1> to <h6>

3) Paragraphs :- Used to display the text using the <p> tag

4) Links :- Used to navigate between webpages using <a> tag

5) Images :- Used to display images using <image> tag

6) Lists :- Used to create ordered and unordered list using <ol>, <ul>,

7) Tables :- Used to organize data in rows & columns

8) Forms :- Used to collect user input using elements such as textboxes, buttons & checkboxes

9) Div & span :- Used to group & organize webpage content

10) Semantic Elements :- Elements such as <header>, <nav>, <section>, <article>, <footer>. Provide meaningful webpage structure.

Css Components :-

1) Selectors :- Select HTML elements to apply styles
2) Properties :- Define the visual characteristic of elements
3) Values :- Specify the setting of a css property
4) Colors :- Define text, background, border colors
5) Fonts :- Control font family, size style and weight
6) Backgrounds :- Set background colors & images
7) Borders :- Define the border style, width and radius
8) Margin :- Controls the space aitside element
9) padding :- Controls the spade inside element
10) Layout :- Control the arrangement of elements using Flexbox, Grid & positioning

HTML VS HTML5

HTML	HTML5
• Older version of HTML	• Latest version of HTML
• Limited Multimedia support	• Supports audio and video
• Uses external plugins for multimedia	• No external plugins are needed
• Has fewer semantic tags	• Provides Semantic tags
• No <canvas> element	• Support <canvas>
• Limited webpage style	• Support local storage
• Complex DOCTYPE declaration	• Simple <!DOCTYPE HTML>

CSS VS CSS3

CSS	CSS 3
• Earlier version of CSS	• Advanced version of CSS
• Basic styling features	• Provides advanced styling features
• Limited animation	• Support animation & transitions
• No flexible layout systems	• Support flexbox & grid
• Limited visual effects	• Supports shadows, gradients, effects.
• No Media queries	• Support Media queries
• Use a single specification	• Divide into different modules

Design : Static Webpages Using HTML & CSS

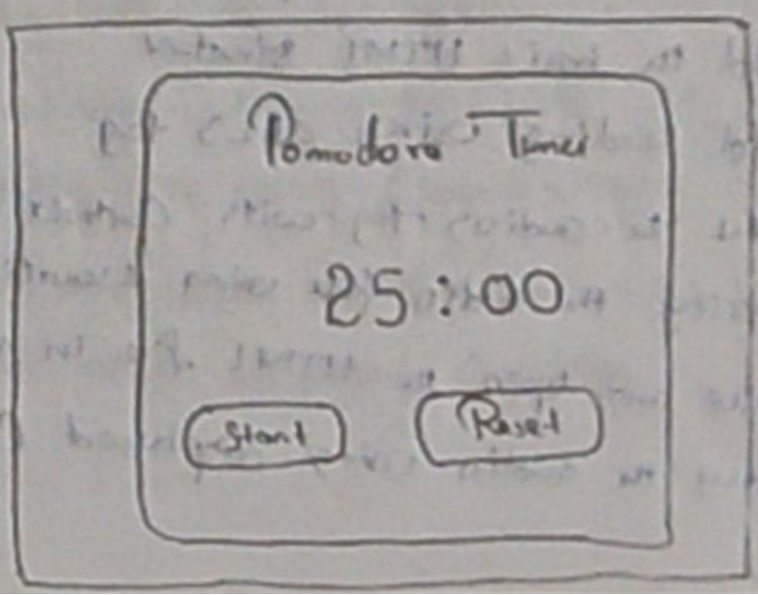
15/7/24

Pomodoro Timer. Using HTML, CSS

```
<!DocType html>
<html>
<head>
    <title Pomodoro Timer</title>
    <style>
        body {
            font-family: Arial;
            background : #ffffff;
            text-align : center;
            Padding : 20px;
        }
        .timer {
            width : 300px;
            Margin : auto;
            Padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 00 10px gray;
        }
        h1 {
            Color : #e74c3c;
        }
        .time {
            font-size : 70px;
            margin : 20px;
```

```
}
button {
    padding : 10px 20px;
    margin : 5px;
    border : none;
    border-radius : 5px;
    background : #e74c3c;
    Color : white;
}
</style>
</head>
<body>
    <div class = "timer">
        <h1> Pomodoro Timer </h1>
        <div class = "time"> 25:00 </div>
        <button> start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

Output :-



Write a Program Using Javascript Array Functions

Code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

Function displayArray(arr) {
    Console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        Console.log(arr[i]);
    }
}

Function findSum(arr) {
    let sum=0;
    for(let i=0; i<arr.length; i++){
        Sum += arr[i];
    }
    return sum;
}
```

```
function changePage() {
document.getElementById("heading").
        innerHTML = "Welcome to javascript!";
document.getElementById("message").
        innerHTML = "The webpage has been changed dynamically";
document.body.style.backgroundColor = "light blue";
    }
</script>
</body>
</html>
```

Output:-

Welcome to My Webpage

Click the button to change this webpage

[Click Me]

Explanation: When user clicks "Click Me", the javascript function changePage() is called. Then the heading, paragraph and background also changes.

After clicking the button:

Welcome to Javascript!

The webpage has been changed dynamically

[Click Me]

Experiment-5

5. Implement modules in Node Js

In Node.js you can implement modules using two different systems: CommonJs (Js), which is traditional default and Esmodule (ESM). The modern Javascript standard.

1. The Common Js Method (default)

CommonJs uses modules. exports to share code and required() to import it. This system loads modules synchronously

(a) Create the module (Math.js):

```
Const add = (a,b) => a+b;
Const Subtract = (a,b) => a-b;
module.exports = {
                    add,
                    Subtract
                  }
```

(b) Use the module (app.js)

```
Const math = required('./math');
Console.log(math.add(10,5));
Console.log(math.subtract(10,5));
```

Output: 15
5

2. The ES modules method (modern standard):

-ES module use export to share code and import to use it -to make Node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

(a) Create the module (Calculator.js):

```
Export Const multiply = (a,b) => a*b
export Const divide = (a,b) => a/b
Const CalculatorName = "Supercate";
Export default CalculatorName
```

(b) Use the Module (main.js)

```
import CalculatorName, {multiply, divide} from
                    'Calculate.js';

Console.log(CalculatorName);
Console.log(multiply(4,2));
Console.log(divide(4,2));
```

Output: Super Cale

8

2

3. Using Built-in Core modules:

Node.js also comes equipped with dozen of built-in Core modules (like for file system or path for file routing) that don't need installation we will load them by modules name:

// Common.Js style:

```
Const fs = required ('fs');
```

// Es module style

```
import fs from 'fs';
```

Here are some of the most commonly used one's;

1. fs = file system Operations
2. https - HTTPS Server and Client
3. Os - operating system utilities
4. path - file path utilities
5. Events - Event handling
6. util - utility functions
7. Stream - stream handling
8. Crpto - Cryptographic functions
9. Url - URL parsing
10. Querystring - URL query String handling

Example :- Simple HTTPS Server

```
http.createServer ( (req, res) => {
    res.writeHead (200, {'Content-Type': 'text/html'});
    res.end ('Hello world!');
}).listen (8080);
```

Output :-

Hello world!

Experiment - 6

6. Develope mini application Using express and Node Js

- Simple mini application Using node Js + Express Js

Student Management System.

1. Add a student
2. View all student
3. delete a student
4. Use Express js router
5. Simple HTML interface

Project Structure:

```
Student-app/
├── server.js
└── public
    └── index.HTML
```

Server.js:

```
Const express = require("express");
Const app = express();
Const PORT = 3000;
app.use(express.json());
app.use(express.static("public"));
app.get("/student", (req, res) => {
                    Res.json(student);
});
// Add
app.post("/student", (req, res) => {
```

```
    Const student = {
    id: students.length + 1;
    name: req.body.name;
    Course: req.body.course
    };
  students.push(student);
  res.json({ message: "student added Successfully",
      student: student });
});
//Delete
app.delete("/students/id", (req, res) => {
    Const id = parseInt(req.params.id);
    students = students.filter(student =>
      student.id !== id);
  res.json({
    message: "student deleted Successfully"});
});
//start Server
app.listen(PORT, () => {
    Console.log('server Running at https://local
    ${PORT}');
});
```

Public/index.html:

```
<!DOCTYPE html>
<html>
```

```
<head>
    <title> Student Management </title>
</head>
<body>
    <h1> Student Management System </h1>
    <input type = "text" id = "name"
    place holder = "Enter Student name">
    <input type = "text" id = "course"
    Place holder = "Enter Course">
    <button on click = "add student()"> Add student
                                        </button>
    <h2> Student list </h2>
    <ul id = "student list"> </ul>
    <script>
        function add student() {
            let name = document.getElementById("name")
                                        .value;
            let course = document.getElementById("course").value;
        fetch("/student", {
            method: "post";
            header: {
                "Content-Type": "application/json"},
            body: JSON.stringify({
                name: name,
                Course: Course,
            })
        })
```

```
        .then (response => response.json())
        .then (data => {
                    alert(data.message());
document.getElementById("name").value = "";
document.getElementById("course").value = "";
    displayStudent();
    });
  }
function displayStudents() {
    fetch("/student") then (response => response.json
  .then (students => {
  let list = document.getElementById("studentList");
  list.innerHTML = "";
students.forEach(student => {
    let li =
      document.createElement("li");
      li.innerHTML = student.id + "_" +
                  student.name + "_" +
                  student.course + "<button
onClick="deleteStudent(" + student.id + ")> Del
</button>";
  list.appendChild(li);
        });
      });
    }
function deleteStudent(id){
    fetch("/student/" + id, {method: "Delete"})
```

```
    .then(response => response.json())
    .then(data => {
        alert(data.message);
        displayStudents();
    });
}
displayStudents();
</script>
</body>
</html>
```

Output:

Student Management System

[Enter student name]

[Enter Course]

[Add student]

Student list:

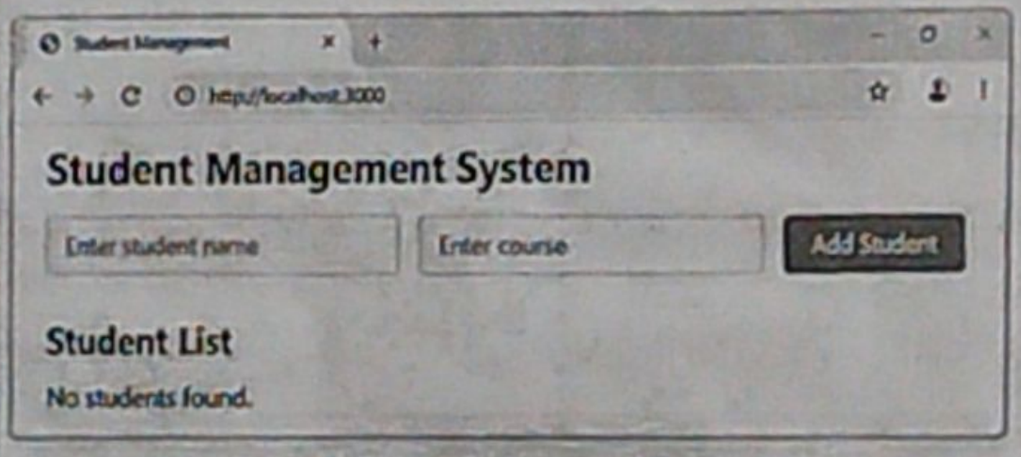
1. Raju - CSM [delete]
2. Ravi - CSE [delete]

- The express.js Server is successfully started at http://localhost:3000

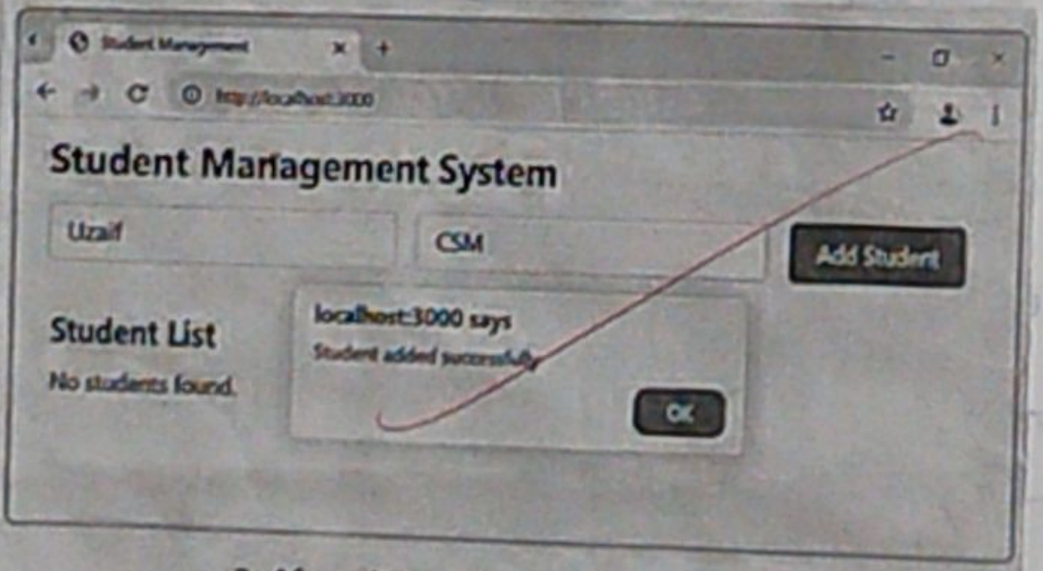
The browser displays the student management system with fields.



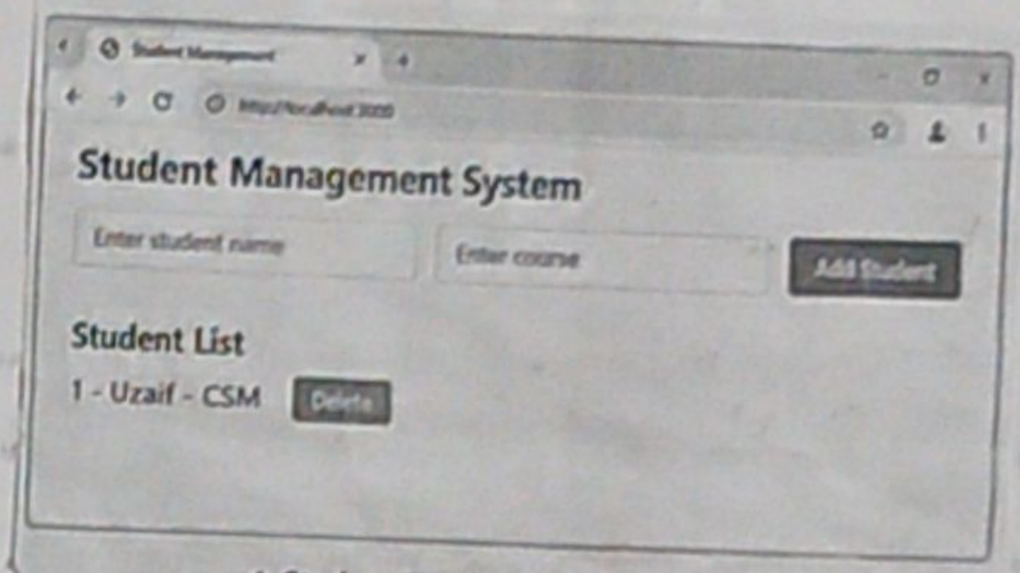
1. Terminal Output (after running 'node server.js')



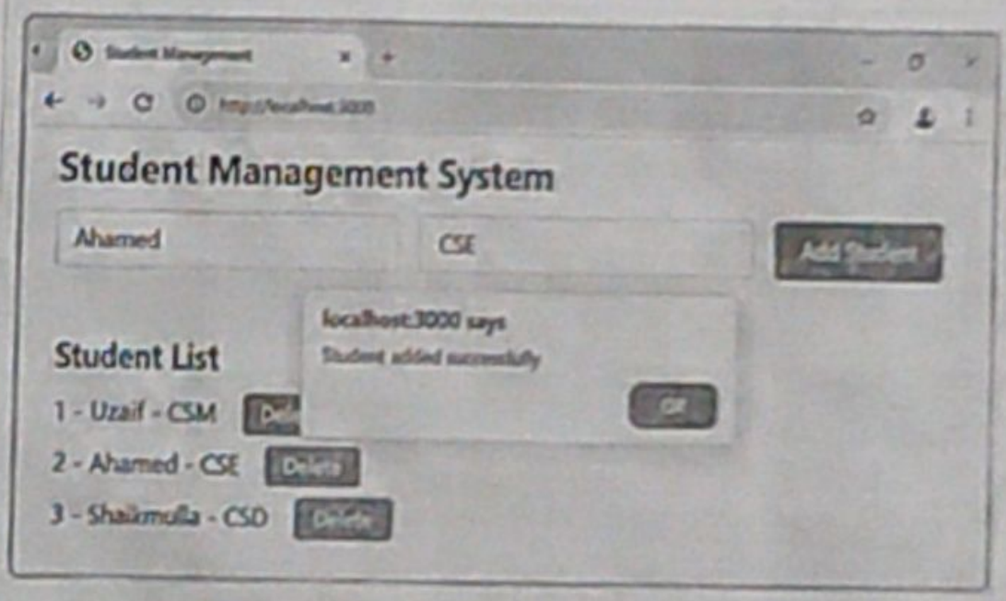
2. Initial Output in Browser (no students)



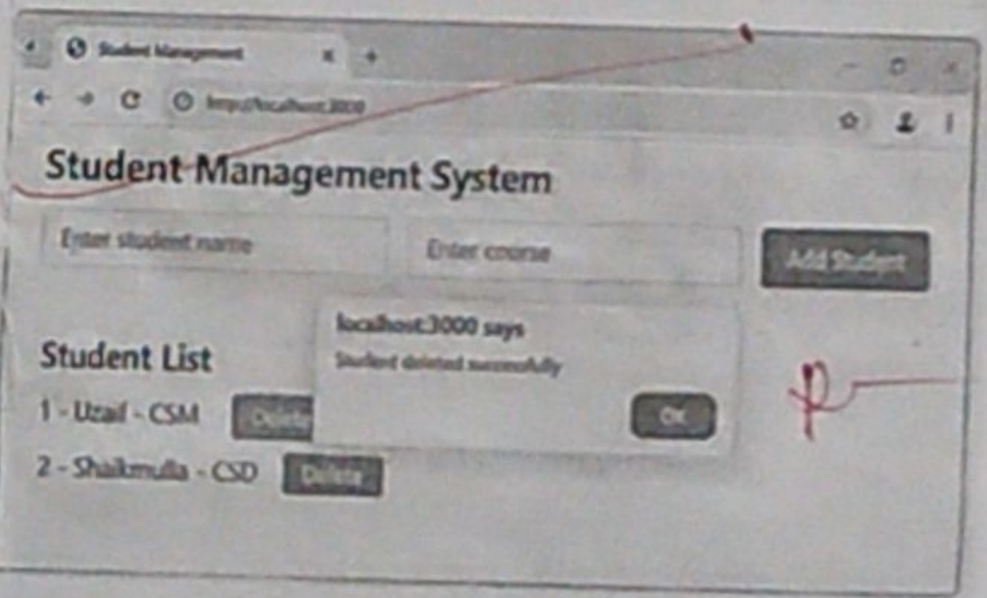
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

```
Const ChangeCourse = () => {
    SetCourse("CSD");
};
Const incrementCount = () => {
    SetCount(count+1);
};
return (
    <div style={{
        border: "2px solid blue",
        Padding: "20px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1> StudentDetail </h1>
    <h2> Functional Component </h2>
    <p> <b> Name: </b> {name} </p>
    <p> <b> Course: </b> {Course} </p>
    <p> <b> Button: </b> {Count} </p>
    <button onClick={changeName}>
        ChangeName </button>
    <button onClick={ChangeCourse}> ChangeCourse </button>
```

Experiment-7

(7) Create a Functional Component and Class Component Using React.

Aim:- To develope a react application using Functional Components to display student details, manage State handle button clicks, and dynamically update data on a webpage.

Functional Component:

A Functional Component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as useState to manage state and update the webpage dynamically

Program:

```
import React, {useState} from
'react';
function StudentFunctional(){
    Const [name, setName] = useState("Tarun");
    Const [course, setCourse] = useState("CSM");
    Const [count, setCount] = useState(0);
    Const changeName = () => {
        SetName("Tejas")
    };
```

```
Const ChangeCourse = () => {
    SetCourse("CSD");
};
Const incrementCount = () => {
    SetCount(Count+1);
};
return (
    <div style={{
        border : "2px solid blue",
        Padding : "20px",
        margin : "20px",
        borderRadius : "10px",
        textAlign : "center"
    }}>
    <h1> StudentDetail </h1>
    <h2> Functional Component </h2>
    <p> <b> Name : </b> {name} </p>
    <p> <b> Course : </b> {Course} </p>
    <p> <b> Button : </b> {Count} </p>
    <button onClick = {changeName} >
        ChangeName </button>

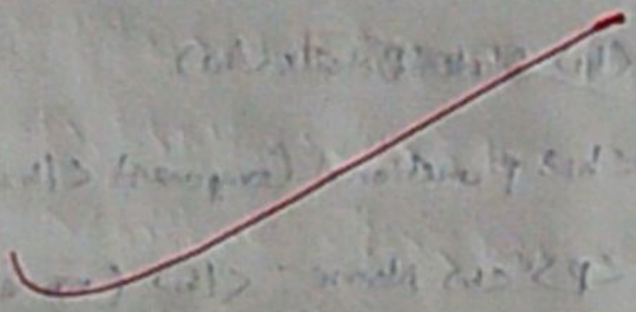
    <button onClick ={ChangeCourse} ChangeCourse </button>
```

<button onClick = {incrementCount}> Increment

</button>

);

Output: When the user clicks the button, the name, Count and Count change dynamically without reading the webpage



Class Component:

-A class Component is an ES6 Javascript class that extends React Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

Program:

```
import React {Component} from
'react';
Class StudentClass extends Component {
    Constructor(props){
        super(props);
        this.state = {
            name: "Tajasaraya",
            Course: "Computer Science"
            Count: 0
        };
    }
    ChangeName = () => {
        this.setState({
            name: "Taja"
        });
    };
    ChangeCourse = () => {
        this.setState({
            Course: "CSM"
        }); });
```

```
incrementCount = () => {
    this.setState((prevState) => {
        Count: prevState.Count + 1
    });
};
render() {
    return (
        <div style={{
            border: "2px solid green",
            Padding: "20px",
            margin: "20px",
            borderRadius: "10px",
            textAlign: "center"
        }}>
            <h1> Student Details </h1>
            <h2> Class Component </h2>
            <p><b>Name: </b> {this.state.name}</p>
            <p><b>Course: </b> {this.state.Course}</p>
            <p><b>ButtonClicked </b> {this.state.Count}</p>
            <button onClick={this.ChangeName}>
                Change Name </button>
            <button onClick={this.ChangeCourse}>
                Change Course </button>
            <button onClick={this.incrementCount}> Increment
                </button>
        </div>
```

Experiment-8

(8). Implement http method using JSON Server

Aim: To implement HTTP methods such as GET, POST, PUT, PATCH, and DELETE using JSON Server to perform CRUD (Create, Read, Update and Delete) operation on Student records)

Theory: JSON Server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operation without developing a backend Server from scratch.

HTTP methods are used to communicate between a client and a server.

- GET – Retrieves data from the server
- POST – Adds new data to the server
- PUT – Replaces an existing record
- PATCH – Updates Select field of a record
- DELETE – Remove a record from the server.

Q. Create a JSON server

Step 1: Initialize the project

Open the Vscode terminal & execute the following Commands.

```
mkdir JSONServer
cd JSONServer
npm init -y
npm instal json server
```

Step 2: create db.json

Create a file named db.json and add the following Student data

```
{
  "student": [
    {
      "id :"1",
      "name" : " Tarun",
      "Course" : "CSM",
      "Mark" : 90,
    },
    {
      "id" : "2",
      "name" : "Tejo",
      "Course" : " CSM",
      "Marks": 90,
    },
    {
      "id" : "3",
      "name": Narayana",
      "Course": "IT",
      "Marks": 81
    }
```

Step 1: start the JSON Server

To the current JSON server version, execute

npx json-server db.json

The server start at:

http://localhost:3000

The student records on the terminal using

http://localhost:3000/students

2. Implementation of HTTP methods

① GET method: The GET method is used to retrieve Student records from JSON Server

Code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

① POST method: The POST method is used to insert a new student record into the server.

Code:

```
fetch ('http://localhost:5000/students', {
method: 'POST',
headers: {
  'Content-Type': 'application/json'
},
body: JSON.stringify({
    name: 'Rahul',
    course: 'CSE',
    marks: 95
})
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
```

```
.catch (error => {
    console.log(error);
});
```

(3) PUT Method: The PUT method is used to replace an existing Student record with update data

Code:

```
fetch ('http://localhost:5000/students/1', {
method: {
    'Content-Type': 'application/json',
},
body JSON.stringify({
    name: 'Tejo',
    Course: 'AIML',
    marks: 90
})
})
.then (response => response.json())
```

```
.then (data => {
    Console.log(data);
})
.catch (error => {
    Console.log(error);
});
```

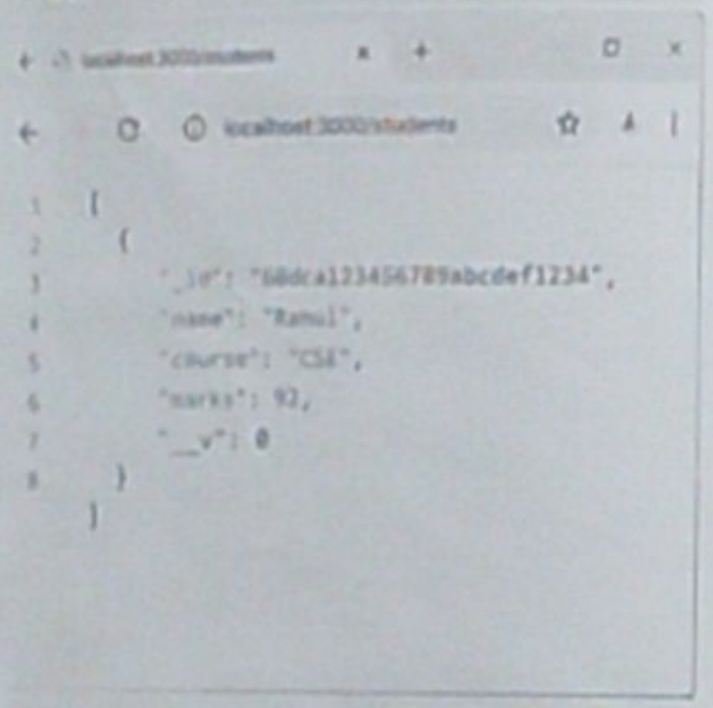
(D.) PATCH Method : The PATCH method is used to update only selected fields of an existing student record

Code: fetch ('HTTP://localhost:3000/students/2', {

```
    method: 'PATCH',
    headers {
      'Content-Type': 'application/json'
    },
    body JSON.stringify ({
      marks: 90
    })
})
.then (response => response.json())
```

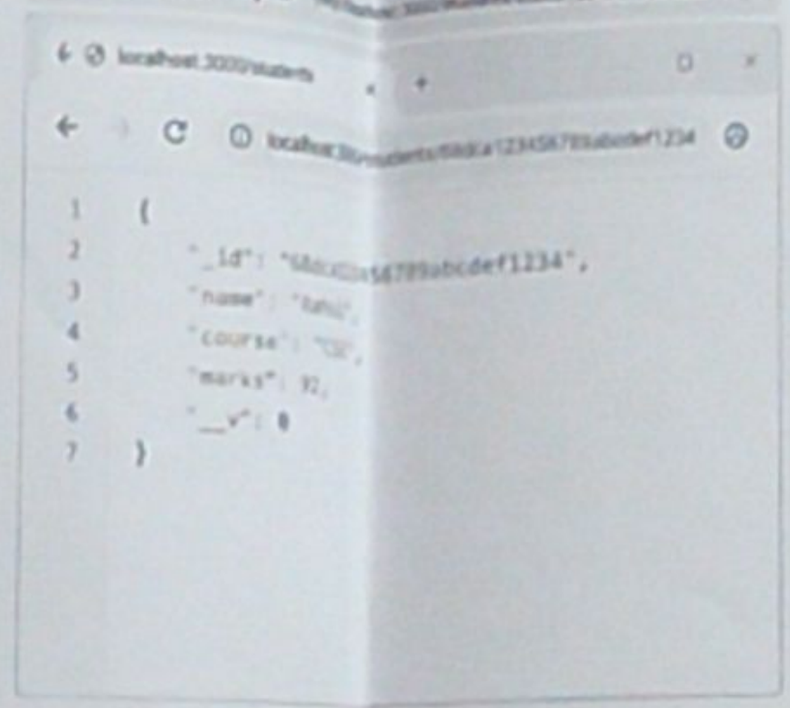
Console.log(data);

1. GET All Students (http://localhost:3000/students)



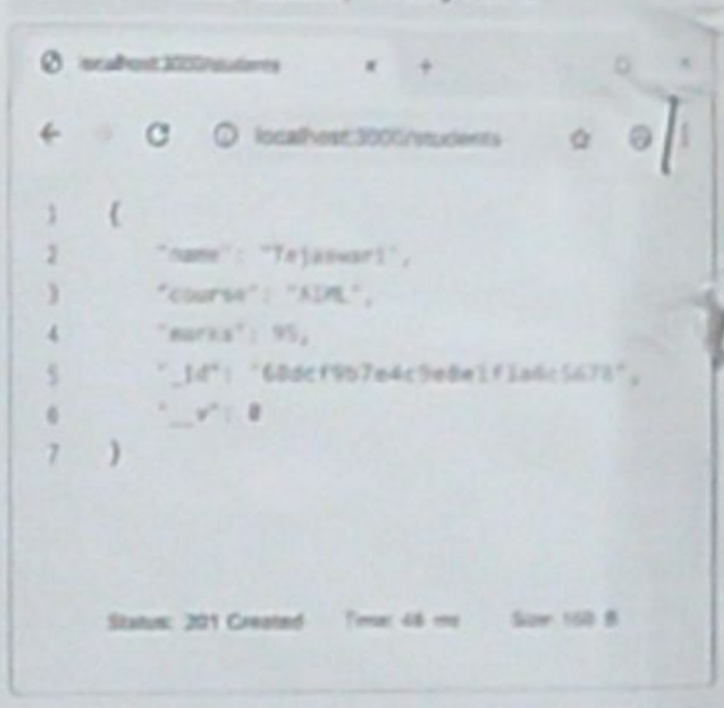
```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```

2. GET Student by ID



```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
}
```

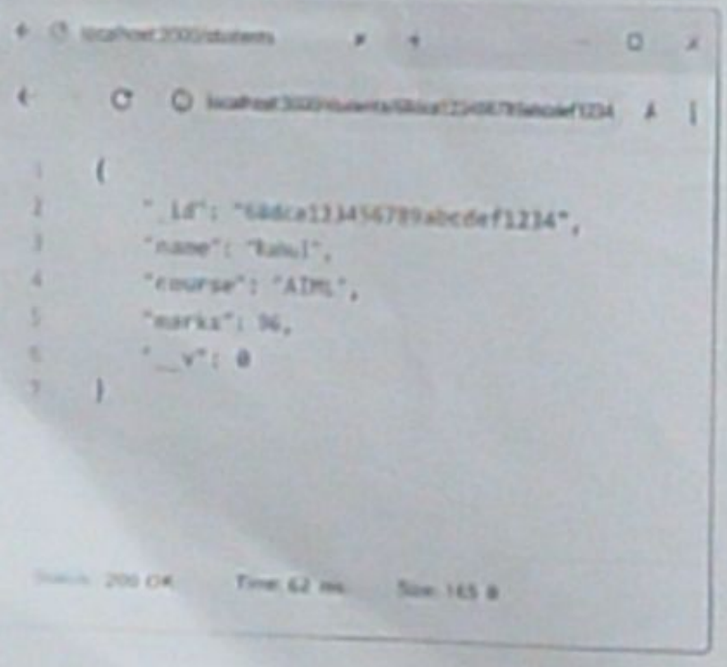
3. POST (Create Student) – Using Postman



```
{
    "name": "Tejaswari",
    "course": "AIML",
    "marks": 95,
    "_id": "68dcf9b7e4c9e8e1f1a6c5d3",
    "__v": 0
}
```

Status: 201 Created Time: 48 ms Size: 168 B

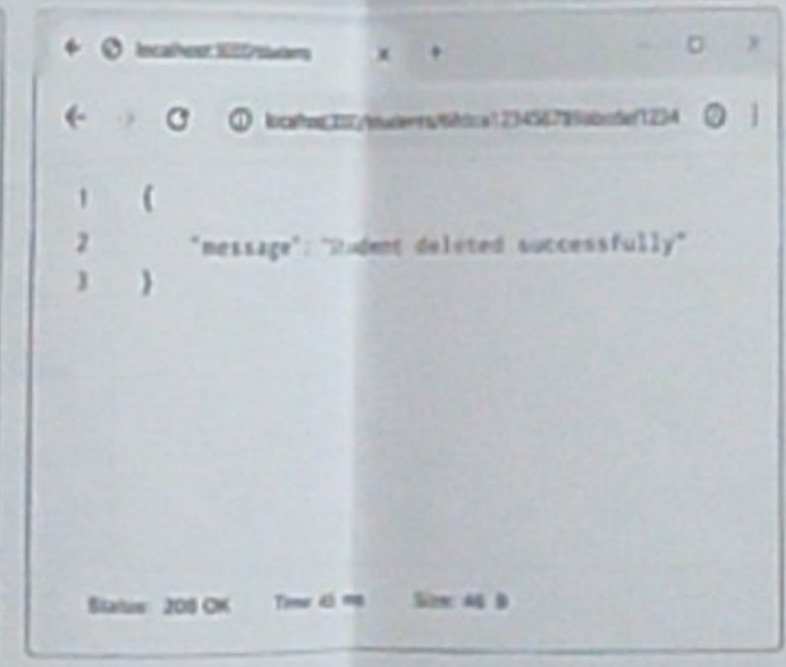
4. PUT (Update Student) – Using Postman



```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "AIML",
    "marks": 96,
    "__v": 0
}
```

Status: 200 OK Time: 62 ms Size: 165 B

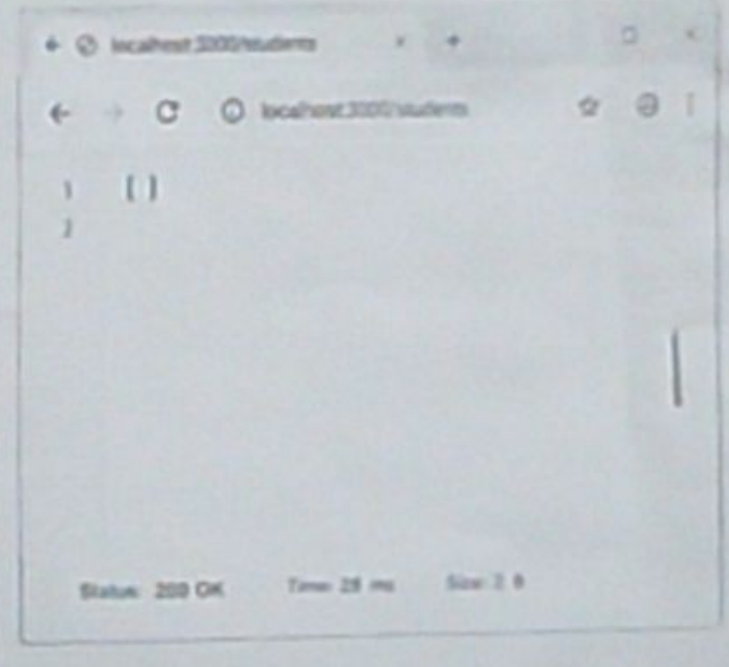
5. DELETE (Remove Student) – Using Postman



```
{
    "message": "Student deleted successfully"
}
```

Status: 200 OK Time: 45 ms Size: 46 B

6. GET All Students (After Deletion)



```
[]
```

Status: 200 OK Time: 28 ms Size: 2 B

- Catch (error) => {
  Console.log(error)
})

Updated, Deleted are called using HTTP requests.

```
Console.log(data);
})
.catch(error => {
    Console.log(error);
});
```

(5) DELETE Method: The DELETE method is used to remove an existing student record from the server.

Code:

```
fetch('http://localhost:3000/student/15', {
    method: 'DELETE'
})
.then(response => {
    if(response.ok) {
        Console.log("student deleted successfully");
    }
})
.catch(error => {
    Console.log(error)
});
```

```
app.use(express.json());
mongoose.connect('mongoDB://127.0.0.1:27017/college')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Create Student
const StudentSchema = new
mongoose.Schema({
    name: {
      type: String,
      required: true
    },
    marks: {
      type: Number,
      required: true
    }
});

//create student model
const Student =
  mongoose.model('Student', StudentSchema);

// create : Add
app.post('/students', async(req,res) => {
  try {
    const student = new Student(req.body);
    const savedStudent = await student.save();
```

Experiment-9

(9) Develope CRUD operation using MongoDB

Aim: To develope a web application using Node.js, express and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON like documents. CRUD operations are used to manage data in database.

MongoOBE is an Object Data Modeling (ODM) library for mongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

Installation and project setup:

```
mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

Program: server.js

```
Const express = require('express')
Const mongoose = required ('mongoose');
Const app = express();
Const PORT = 3000;
```

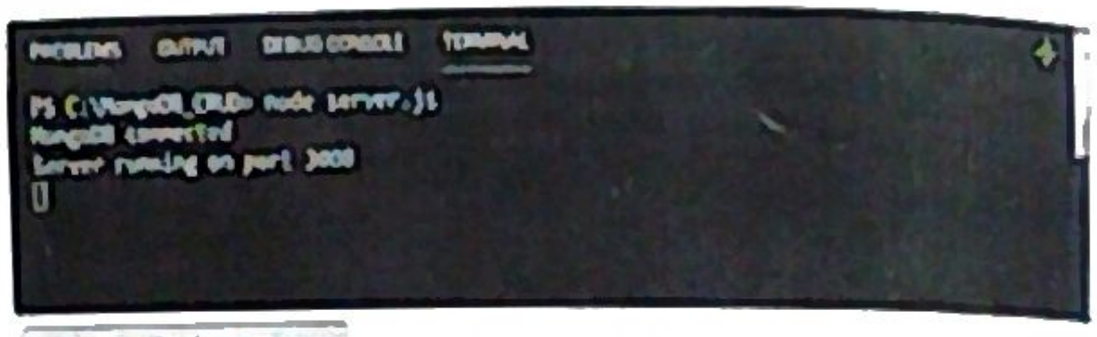
```
res.status(201).json(savedStudent);
} catch(error) {
    res.status(400).json({message error: message});
    }
});
//Read : get all students
app.get('/students', async(req,res) => {
    try {
        const student = await
        student.find();
        res.status(200).json({message error message});
    }
});
//READ : get student by ID
app.get('/students/:id', async(req,res) => {
    try {
        const student = await
        student.findById(req.params.id)
        if (!student) {
            return res.status(404).json({
                message: "student not found"
            });
        }
        res.status(200).json(student)
    } catch(error) {
```

```
        res.status(500).json({ message:
        error.message});
    }
});
//UPDATE
app.put('/student/:id', async(req,res)=>{
    try{
    const student = await
    Student.findByIdAndUpdate(
        req.params.id,
        req.body
        {new:true, runValidators:true}
    );
    if(!student){
        return res.status(404).json({
            message:"student not found"
        });
    }
    res.status(200).json(student);
    }catch(error){
     res.status(500).json({message:
        error.message});
    }
});
//DELETE
```

```
//DELETE
app.delete('/student/:id', asyn (req,res) => {
    try {
        Const student = await student
            findById AndDelete(
        req.params.id
        );
        if (!student) {
            return res.status(404).json({
                message: "student not found"
            });
        }
        res.status(200).json({
            message: "student deleted Successfully"
        });
    } Catch error) {
        res.status(500).json({message:
            error.message});
    }
});
```

//DELETE

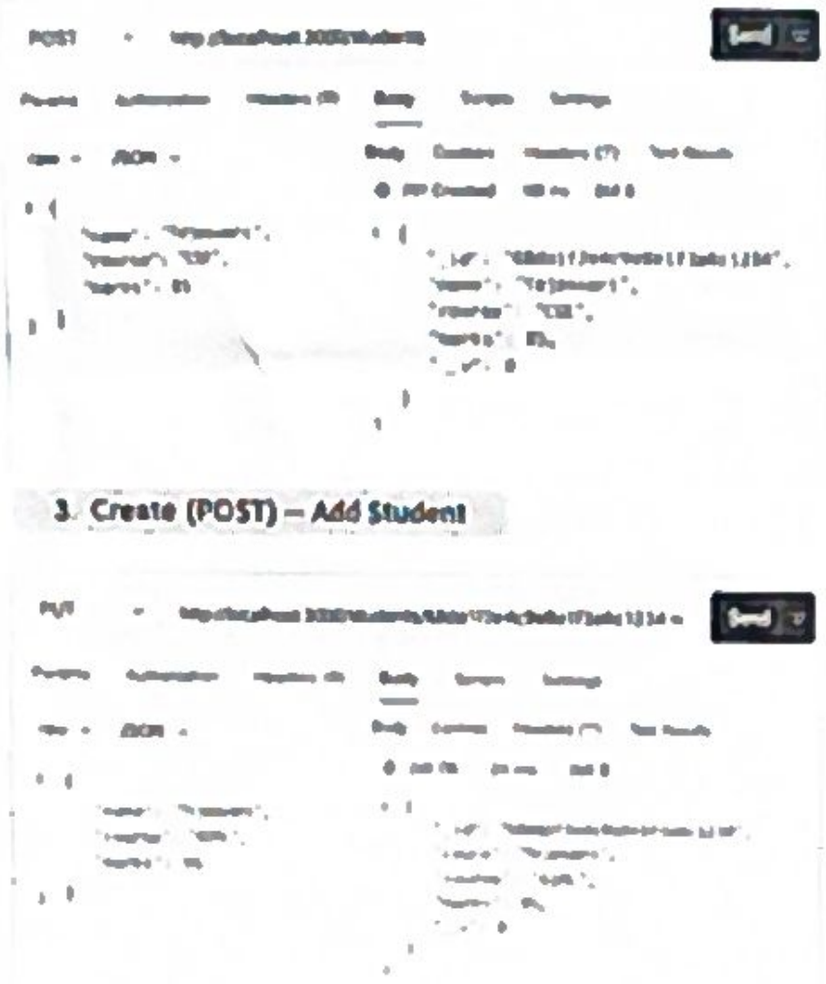
app.delete('/students/:id', async (req, res) => {



1. Terminal Output



2. Chrome Browser (Home Page)



3. Create (POST) – Add Student

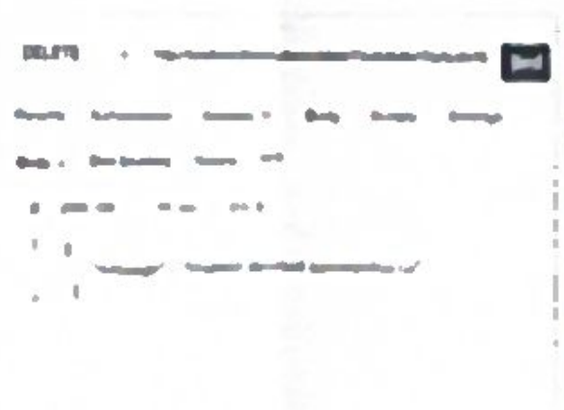


4. Read (GET) – Get All Students

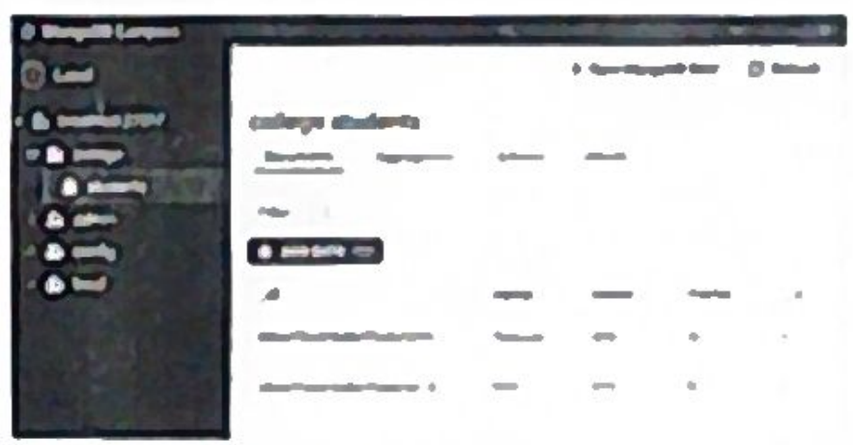


5. Read (GET) – Get Student by ID

6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View